

App Dev Project Report

1. Student Details

Name: Konda Nitish

Roll Number: 23f2000490

Email: 23f2000490@ds.study.iitm.ac.in

About Me: I am a student in the IITM BS Degree program, interested in the Domains of Ai/ML and Data Science and always intrested in learning new Technologies and exploring new Domains in the field of Computer Science.

2. Project Details

Project Title: Trekking Management Application

Problem Statement:

Adventure organizations relying on spreadsheets and manual coordination struggle to manage trek approvals, staff assignments, and bookings. This project builds a role-based web application for Admins, Trek Staff, and Trekkers to manage the entire trekking lifecycle digitally.

Approach:

The app was built with Flask following a blueprint-based modular structure (auth, admin, staff, user), with SQLAlchemy models defining Users, Treks, Bookings, and Staff Profiles. Each role gets a dedicated dashboard with server-side rendered views via Jinja2, enforcing role-based access, staff approval workflows, and booking rules (no overbooking, no duplicate bookings) directly in the Flask route logic.

3. AI/LLM Declaration

I used **Claude (Anthropic)** to assist in scaffolding a few Flask routes and blueprints, debugging occasional routing and template errors, and suggesting some Bootstrap UI structure.

The extent of AI/LLM usage is around **10%**, limited to **code suggestions, debugging assistance, and UI structuring**.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
Flask-SQLAlchemy	ORM for SQLite database
SQLite	Lightweight relational database
Jinja2	Server-side template rendering
Bootstrap 5	Responsive frontend styling
Flask-Login	Session management and authentication
Werkzeug	Password hashing

5. Database Schema / ER Diagram

Tables:

User - stores user login and profile details (id, name, email, password_hash, role, contact, is_blacklisted, is_approved, created_at)

StaffProfile - stores staff-specific approval status (id, user_id, status)

Trek - stores trek details (id, name, location, difficulty, duration, description, available_slots, total_slots, assigned_staff_id, status, start_date, end_date)

Booking - records a user's trek booking (id, user_id, trek_id, booking_date, status)

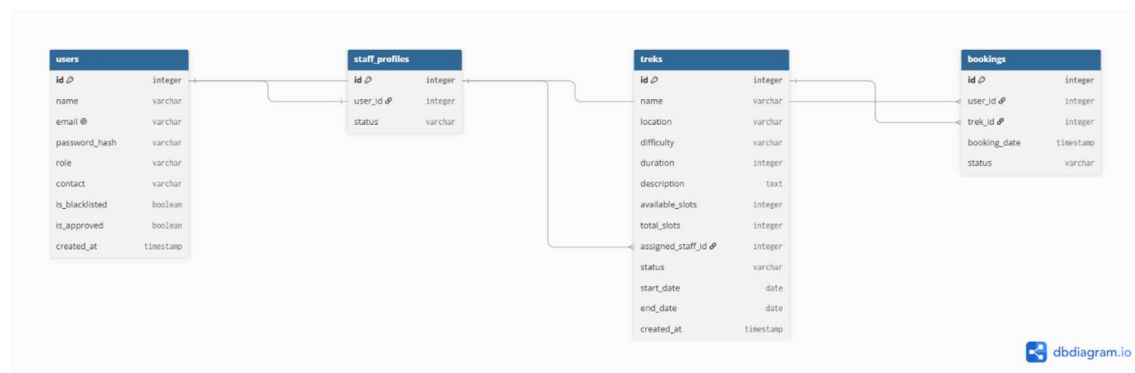
Relationships:

One-to-One → User → StaffProfile

One-to-Many → StaffProfile → Trek

One-to-Many → User → Booking

One-to-Many → Trek → Booking



6. Architecture and Features

Architecture Overview:

- app.py – main Flask application entry point, app factory and blueprint registration
- models.py – database models using SQLAlchemy (User, StaffProfile, Trek, Booking)
- auth.py, admin.py, staff.py, user.py – Flask Blueprints for authentication and role-based routes
- /templates – Jinja2 HTML templates, organized by role (admin/, staff/, user/)
- /static – CSS files for custom styling on top of Bootstrap

Implemented Features:

- Role-based registration and login for Admin, Trek Staff, and Trekkers
- Trek Staff approval and blacklist workflow
- Trek CRUD with staff assignment
- Booking system with duplicate and overbooking prevention
- Auto-completing bookings tied to trek status
- Dedicated trekking history view
- Search across treks, staff, and users
- Responsive, mobile-friendly UI built with Bootstrap
- Secure password hashing and session-based authentication

Additional Features:

- Slot-aware trek editing that automatically adjusts available slots when total slots change
- Status-based booking filter on the Admin bookings view

7. Video Presentation

Drive Link: <https://drive.google.com/file/d/1v1vE1JCxwwAN86C0ebLASCfClFIkNFLW/view?usp=sharing>